Delta Lake

Delta Lake is an open-source storage layer that adds reliability to data lakes by enabling ACID transactions on top of Parquet files. It uses a Delta Log alongside Parquet data files to track changes, ensuring data consistency even with concurrent writes or failures. This supports features like time travel for querying historical versions and schema evolution without breaking pipelines.

What is unity catalog

Unity Catalog is Databricks' unified governance solution for managing and securing data across multiple workspaces. It acts as a central catalog that simplifies data discovery, access control, and tracking.

Data Lineage

Data Lineage tracks the **origin, movement, and transformations** of data from its source to its ultimate destination. In Azure Databricks, this is automatically captured at a granular level (tables, views, columns, and notebooks) using **Unity Catalog**.

Auto Loader

Auto Loader in Databricks simplifies incremental file ingestion into Delta Lake tables by automatically detecting and processing new files from cloud storage (like ADLS Gen2) without manual tracking of processed files.

Managed vs External

managed tables and external tables differ primarily in how data lifecycle, storage, and governance are handled. Managed tables are fully controlled by Unity Catalog, which automatically manages both the metadata and the underlying data files. when you drop a managed table, both metadata and data are deleted. External tables, by contrast, only manage metadata within Unity Catalog while referencing data stored in a user-specified external location. dropping the table removes only the metadata, leaving the actual files untouched

Unity metastore vs hive metastore

Unity Catalog and Hive Metastore both manage metadata for tables and schemas in Databricks, but Unity Catalog offers centralized, enterprise-grade governance while Hive Metastore is a legacy, workspace-local solution. Unity Catalog operates at the account level with a single metastore governing multiple workspaces, enabling cross-workspace sharing, lineage, and policies. Hive Metastore is confined to individual workspaces and requiring manual replication for multi workspace setups

Schema evolution In delta lake

Schema evolution in Delta Lake allows the table schema to automatically adapt when new data with additional columns or compatible type changes arrives during writes, without breaking existing queries or requiring manual intervention. By default, Delta Lake enforces strict schema validation on writes to ensure data quality. To enable evolution, we use the 'mergeSchema' option set to 'true' during append operations.

All-Purpose vs. Job Clusters:

All-purpose (interactive) clusters are long-running and manually managed for ad-hoc, collaborative development. Job clusters are automatically provisioned strictly to run a specific task and terminate immediately, making them highly cost-effective for production.

Standard vs. High Concurrency Mode:

Standard mode is for single-user analytical workspaces. High Concurrency allows multiple users to share a cluster with resource isolation and granular security, like row-level permissions.

Delta Lake vs. Parquet:

Parquet is a plain, open-source file format. Delta Lake is built on top of Parquet. It includes a transaction log that adds ACID transactions, time travel, schema enforcement, and upsert/delete capabilities.

Databricks vs. Apache Spark:

Apache Spark is a free, open-source distributed computing engine. Databricks is a fully managed Software-as-a-Service (SaaS) platform built on top of Spark. It adds optimized runtimes, built-in governance, collaborative notebooks, and serverless capabilities.

Databricks vs. Azure Data Factory (ADF):

ADF is primarily an orchestration and ETL ingestion tool with a drag-and-drop interface for connecting different systems. Databricks is a heavy-duty computing and analytics engine used for complex code-based transformations, machine learning, and advanced AI.

DataFrame vs. RDD (Resilient Distributed Dataset):

DataFrame is a high-level API with data organized into named columns, allowing Spark to optimize queries using Catalyst. RDD is the low-level, schema-less API that handles raw Python objects, offering full control but losing all automatic optimization benefits.

Spark DataFrame vs. Pandas DataFrame:

Spark DataFrame processes data across a distributed cluster of nodes, making it built for big data. Pandas DataFrame operates entirely on a single machine's CPU/RAM, making it much faster for small datasets but prone to crashing on large ones.

Transformations vs. Actions:

Transformations (like .select(), .filter(), .join()) define the blueprint of your execution plan but are lazy and don't process data immediately. Actions (like .show(), .collect(), .count()) trigger the actual computation of all previous transformations to return a result.

Narrow vs. Wide Transformations:

Narrow transformations (like .map(), .filter()) keep data within the same partition, requiring no data movement across nodes. Wide transformations (like .groupBy(), .join()) force data shuffling across the network to group related data together, which is highly resource-intensive.

.cache() vs. .persist():

Both store dataframes in memory for fast reuse. .cache() uses the default storage level (MEMORY_AND_DISK), while .persist() allows you to customize the storage level (e.g., MEMORY_ONLY, DISK_ONLY, or serialized versions).

coalesce() vs. repartition():

Both change the number of data partitions. coalesce() decreases partitions without a full data shuffle by merging adjacent partitions. repartition() can increase or decrease partitions and forces a full network shuffle to distribute data evenly.

User Defined Functions (UDFs) vs. Pandas UDFs (Vectorized UDFs):

Standard UDFs evaluate data row-by-row, which is slow because Spark must serialize data between the JVM and Python engine for every row. Pandas UDFs use Apache Arrow to transfer and execute data in vectorized batches, drastically boosting performance.

Data Skew

Data skew happens when data is **unevenly distributed** across cluster partitions. During operations like joins or aggregations, one or two executors end up processing a massive chunk of the data while the others sit idle, bottlenecking the entire pipeline.

Liquid Clustering

Liquid Clustering is Databricks' **next-generation data layout technique** that replaces traditional Hive partitioning (PARTITIONED BY) and Z-Ordering. It dynamically adjusts the data layout based on query patterns without requiring rigid, pre-defined physical directories.

Data Masking

Data masking is a security technique used to obscure sensitive information (like PII—Personally Identifiable Information, SSNs, credit cards) while keeping the data format intact. In Azure Databricks, this is primarily implemented using **Dynamic Data Masking (DDM)** via **Unity Catalog**, which applies column-level security policies on the fly without altering the raw underlying data.

Partition BY vs. Z-ORDER

Both PARTITION BY and Z-ORDER are data layout techniques used to optimize read performance in Delta Lake by reducing the amount of data scanned (**Data Skipping**), but they operate on completely different levels.

Key Interview Points

- **PARTITION BY (Physical Layout):** * **Mechanism:** Physically splits data into separate, distinct sub-directories on disk based on a specific column value.
    - **Best Use Case:** Low-cardinality columns (e.g., Year, Month, Region) where data is evenly distributed.
    - **Anti-Pattern:** High-cardinality columns (e.g., User_ID, Timestamp). This creates millions of tiny directories and files, destroying storage metadata and query performance (**Over-partitioning**).
- **Z-ORDER (Logical Layout / Clustering):** * **Mechanism:** Co-locates related information within the *same* file or group of files using a space-filling curve algorithm. It does not alter directories.
    - **Best Use Case:** High-cardinality columns frequently used in WHERE clauses (e.g., Customer_ID, Product_ID).
    - **Flexibility:** Can be applied to multiple columns to optimize queries that filter on various combinations of attributes.
- **Coexistence:** You can combine them. You can physically partition a table by a low-cardinality column (Country), and then run a Z-ORDER BY on a high-cardinality column (User_ID) within those partitions.

OPTIMIZE vs. VACUUM

Both OPTIMIZE and VACUUM are critical maintenance commands in Delta Lake used to improve query performance and manage storage costs, but they perform opposite file-management functions.

Key Interview Points

- **OPTIMIZE (File Compaction):** * **Purpose:** Resolves the "small file problem" by merging small files into larger, uniform files (typically ~1 GB).
    - **Performance Impact:** Drastically speeds up read queries by reducing network overhead and file open/close seek times.
    - **Mechanism:** It does not delete old files; it creates new compacted files and updates the Delta transaction log.
- **VACUUM (Garbage Collection):** * **Purpose:** Permanently deletes files that are no longer referenced by the Delta table log and are older than a specific retention threshold.
    - **Storage Impact:** Frees up space and reduces costs in cloud storage (ADLS).
    - **Retention Period:** The default retention period is **7 days (168 hours)**. To run it for shorter durations, you must disable a safety check (spark.databricks.delta.vacuum.parallelDelete.enabled or data retention check overrides).
    - **Risk:** Running VACUUM breaks time travel to versions older than the retention threshold.

Database Normalization

Normalization is the process of organizing data in a database to **reduce data redundancy** and **eliminate data anomalies** (insertion, update, and deletion anomalies). It involves breaking a large, flat table into smaller tables and defining relationships between them.

Key Interview Points

- **The Goal:** Ensure that every non-key column depends strictly on the primary key, minimizing duplicated storage and data contradictions.
- **1NF (First Normal Form):** Atomic values only. No repeating groups or arrays in a single column.
- **2NF (Second Normal Form):** Must be in 1NF, and all non-key attributes must fully depend on the entire primary key (eliminates partial dependencies in composite keys).
- **3NF (Third Normal Form):** Must be in 2NF, and no non-key attribute can depend on another non-key attribute (eliminates transitive dependencies). **This is the standard target for relational database design.**